

ENTRENAMIENTO PARA DESAFÍO II 2026-1

INFORMÁTICA 2

FACULTAD DE INGENIERÍA
UNIVERSIDAD DE ANTIOQUIA



1. Introducción

Una lista enlazada es una estructura de datos lineal y dinámica, compuesta por nodos secuenciales no contiguos en memoria. La lista es una secuencia de n elementos, con $n \geq 0$; puede contener elementos repetidos y, si $n = 0$, se dice que la lista está vacía. Explícitamente, no se almacenan posiciones en una lista enlazada, pero implícitamente se pueden implementar operaciones basadas en la posición de cada nodo en la lista. En consecuencia, suele caracterizarse la posición del primer y del último elemento de la lista.

Las listas enlazadas son eficientes para gestionar la memoria dinámicamente, pero pueden ser lentas en el acceso aleatorio. Sin embargo, según su implementación, pueden permitir inserciones y eliminaciones de forma eficiente.

En esta práctica se aborda el diseño e implementación de una lista enlazada utilizando plantillas, apuntadores y programación orientada a objetos en C++. Posteriormente, esta estructura se emplea como base para la construcción de una matriz dinámica representada mediante listas enlazadas.

2. Objetivos

2.1. Objetivo general

Diseñar e implementar una lista enlazada genérica en C++, utilizando plantillas, apuntadores y programación orientada a objetos, y emplearla como base para representar estructuras de datos más complejas.

2.2. Objetivos específicos

- Implementar la clase `nodo` con un campo de datos genérico y un apuntador al siguiente nodo.
- Desarrollar la clase `lista` que gestione dinámicamente los nodos mediante apuntadores.
- Implementar las operaciones básicas de manipulación de la lista enlazada, considerando criterios de eficiencia.
- Utilizar la lista enlazada como estructura base para la construcción de una matriz dinámica.
- Aplicar principios de programación orientada a objetos en el diseño de las clases.

3. Diseño e implementación (usando apuntadores y POO en C++)

3.1. Estructura de datos

Primero, se debe crear la clase `nodo` basada en un tipo genérico `T`. Cada nodo tiene dos campos: el campo de datos, que debe declararse de acuerdo con el parámetro de la plantilla `T`, y un apuntador a

nodo (**ptrNext**) que permitirá enlazarlo con el nodo siguiente. Luego se crea la clase **lista**, que debe contener como mínimo un apuntador a un nodo.

La estructura de la clase **nodo** se ilustra en la Figura 1.

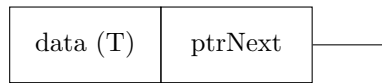


Figura 1: Diagrama de la clase nodo

La organización de la lista enlazada completa se muestra en la Figura 2. La lista inicia con dicho apuntador. A partir de allí, cada nodo se enlaza con el siguiente a través del apuntador a nodo (siguiente). El apuntador del último nodo de la lista debe apuntar a nulo.

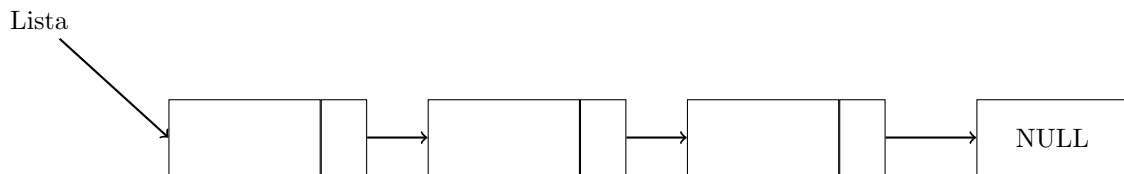


Figura 2: Diagrama de una lista enlazada

3.2. Operaciones de la clase lista

Las operaciones que debe soportar la clase **lista** se presentan a continuación:

- Construir una lista vacía
- **tamano()**: Devuelve el tamaño de la lista
- **esVacia()**: Verifica si la lista está vacía
- **primero()**: Devuelve el primer elemento de la lista (head)
- **ultimo()**: Devuelve el último elemento de la lista
- **agregar(e, i)**: Agrega un elemento **e** en la posición **i**
- **eliminar(e)**: Elimina la primera aparición del elemento **e** en la lista. El elemento **e** debe corresponder al tipo **T**.
- **consultar(i)**: Devuelve el dato en la posición **i**
- **reemplazar(e, i)**: Reemplaza el elemento en la posición **i**
- **memoryUsage()**: Devuelve la memoria ocupada (usar atributo **static**)
- **concatenar(B)**: Concatena la lista **B** al final de la lista actual
- Destruir lista

Tenga presente el criterio de eficiencia al implementarlas. Puede añadir operaciones adicionales de manera justificada.

3.3. Implementación

En la primera sesión de su entrenamiento debe implementar la clase lista enlazada, utilizando plantillas, apuntadores y POO en C++.

4. Uso de la clase lista enlazada

4.1. Clase `matrizFloat`

Cree la clase `matrizFloat` para representar una matriz de dimensiones $N \times M$ de números `float`.

La clase `matriz` representa un arreglo dinámico que contiene tantas posiciones N como filas va a tener la matriz. En consecuencia, los elementos de este arreglo dinámico serán simplemente listas enlazadas. Para esta etapa de uso no modifique su implementación previa de la clase `lista`; compórtese únicamente como usuario de ella.

4.2. Operaciones de la clase `matriz`

Las operaciones que debe soportar la clase `matrizFloat` son las siguientes:

- Construir matriz: Lee y carga una matriz $M \times N$ desde un archivo `.txt`. El archivo a leer sólo contiene los datos así: cada fila en una línea diferente del archivo y los datos dentro de cada fila (correspondientes a las columnas) separados por un espacio en blanco.
- Imprimir matriz: Imprime la matriz en pantalla
- Eliminar fila: Elimina la fila `i` de la matriz
- Destruir lista

5. Notas

Las entregas basadas en generadores automáticos de código no se considerarán válidas. Considere todas las validaciones/excepciones necesarias durante el desarrollo.